LifeLedger

Your Health. Your Keys. Your Life.

A Web3 Lifelong Patient Health Record System
built on Account Abstraction, Decentralized Storage & Smart Contracts

Technical Blueprint & Pitch Deck Foundation

Track: Web3 / HealthTech — Stack: Solidity, Next.js, ERC-4337, IPFS — Chain: Polygon Amoy / Base Sepolia

Contents

1 Executive Summary **3**

2 The Problem vs. The Solution **3**
2.1 The Problem: Web2 Healthcare Is Fundamentally Broken . . . 3
2.2 The Solution: A Patient-Owned Cryptographic Record . . . 4

3 Design Lineage & Key Differentiators **4**
3.1 From the Healthchain Paper (Academic Base) . . . 5
3.2 From the Core LifeLedger Design . . . 5
3.3 Our Additions on Top of Both . . . 5

4 Complete Feature Set **6**
4.1 Identity & Custody Lifecycle . . . 6
4.2 Vaccination Soulbound Tokens (SBTs) . . . 6
4.3 The Doctor Visit Flow (the demo's centerpiece) . . . 6
4.4 Encryption & Storage . . . 7
4.5 Access Control & Consent . . . 7
4.6 GDPR — "Crypto-Shredding" Right to Erasure . . . 7
4.7 Provenance / Audit Trail . . . 7
4.8 Adult Onboarding & Legacy Record Migration . . . 7
4.9 Prescriptions & Pharmacist Flow . . . 8
4.10 E-Referrals & Out-of-Network Sharing . . . 8
4.11 Additional Record Granularity . . . 8
4.12 Insurance Claim Verification — The Last Hook . . . 9

5 Core User Journeys **10**
5.1 Phase 1 — Custodial Management (Birth to Age 13) . . . 10
5.2 Phase 2 — Co-Management (Age 13 to 18) . . . 10
5.3 Phase 3 — Independence (Age 18+) . . . 11
5.4 The Doctor Visit Flow . . . 11
5.5 Adult Migrating from Paper Records . . . 12
5.6 Pharmacist . . . 12
5.7 Receptionist (Stretch) . . . 12
5.8 Insurer . . . 13

6 Technical Architecture & Tech Stack **14**
6.1 System Architecture Overview . . . 14
6.2 How a Record Moves Through the System . . . 14
6.3 Record Types & Provenance . . . 15
6.4 Frontend . . . 15
6.5 Wallet, Auth & Account Abstraction . . . 15
6.6 Decentralized Storage & Encryption . . . 16
6.7 Blockchain Layer . . . 16
6.7.1 Key Smart Contracts & Functions . . . 16
6.7.2 Extended Contract Additions (the "Tadka" Layer) . . . 18
6.7.3 Supporting Infrastructure . . . 19
6.8 Full Tech Stack . . . 19

7 Repository Structure & Parallel Work Split **21**
7.1 Repository Layout . . . 21
7.2 The Parallel Work Split . . . 21
7.3 Git Workflow . . . 23

8 Deployment & Domain Setup **23**

9 Hurdle Mitigation **25**
9.1 "What happens when someone loses their private key?" . . . 25
9.2 "How do you comply with GDPR's Right to Be Forgotten on an immutable blockchain?" . . . 25

10 Open Design Decisions **26**

11 Pitch & Demo Script **27**

12 Future Scope **28**

1 Executive Summary

Modern healthcare data is trapped. Every time a patient is born, moves cities, switches insurance providers, or simply changes pediatricians, their medical history fractures into another disconnected silo — a proprietary hospital EHR (Electronic Health Record) system that does not talk to the next one. The result is a $30+ billion global patchwork of centralized databases, each one a high-value ransomware target, each one a single point of failure, and none of them actually owned by the patient whose life depends on the data inside them.

LifeLedger rebuilds the patient record from first principles: instead of a hospital owning a copy of your data, you own the only copy that matters — a cryptographically verifiable, end-to-end encrypted, patient-controlled record that travels with you from your first vaccination to your last checkup, across every doctor, hospital, and border you ever encounter.

What makes LifeLedger a genuine innovation rather than "healthcare, but with a blockchain bolted on" is its treatment of time and custody as first-class protocol concepts. A newborn cannot hold a private key, so LifeLedger does not force them to. Instead, it introduces a progressive sovereignty model: a mother or guardian custodially manages the child's wallet and record from birth, doctors mint verifiable Vaccination Soulbound Tokens (SBTs) directly to that wallet, and — with zero manual intervention — a smart contract autonomously executes a cryptographic ownership handover the moment the child turns 18, permanently revoking parental access and granting the now-adult full, independent control of a complete, lifelong medical history.

On top of that core loop, LifeLedger layers a second differentiator: the record system does not assume every patient starts as a newborn. An adult who has only ever kept paper records can self-onboard directly into full independence, migrate their own history, and have it progressively upgraded from self-reported to clinician-verified as they see real doctors. A scoped, category-level (and eventually record-level) access-grant model means a doctor, pharmacist, or insurer only ever sees the exact slice of a patient's history they were explicitly authorized to see — never the whole eighteen-year record by default.

This document lays out the full technical architecture, the complete feature set, and the pitch narrative for taking LifeLedger from empty repo to a working, demoable product.

2 The Problem vs. The Solution

2.1 The Problem: Web2 Healthcare Is Fundamentally Broken

Healthcare IT infrastructure was built in an era before cyberattacks were common, before patients moved between providers dozens of times in a lifetime, and before anyone imagined a global pandemic that would demand instant, verifiable proof of vaccination. Four structural flaws define the status quo:

- **Centralized Honeypots.** Every hospital, clinic, and insurance network operates its own siloed SQL/NoSQL database of millions of patient records. A single breach exposes everything at once — there is no cryptographic compartmentalization, only perimeter security that regularly fails.
- **Ransomware Epidemic.** Healthcare is now the single most targeted industry for ransomware globally. Because records are centrally stored and centrally administered, a single successful attack can lock an entire hospital network out of patient charts, delaying care and, in documented cases, costing lives.

- **The Lost Paper Trail.** Vaccination cards, immunization booklets, and physical referral letters are still the primary source of truth in huge parts of the world. They get lost, water-damaged, or simply forgotten — and when they're gone, so is the proof.
- **Fragmentation on Every Provider Switch.** Changing pediatricians, moving to a new city, or switching insurance almost always means starting from a fragmented, incomplete history. Interoperability standards like HL7/FHIR exist, but adoption is inconsistent and patients still don't own or control the data — providers do.

Underlying every one of these problems is the same root cause: **the patient is not the custodian of their own data. Hospitals are.**

2.2 The Solution: A Patient-Owned Cryptographic Record

LifeLedger inverts the trust model entirely. Instead of a hospital's database being the source of truth, the source of truth becomes a combination of on-chain cryptographic proofs and client-side-encrypted decentralized storage, both of which are controlled exclusively by keys the patient (or their guardian, temporarily) holds.

- **Client-Side Encryption Before Anything Touches the Network.** Every medical record — diagnosis notes, lab results, imaging metadata — is encrypted in the browser/app using the patient's public key before it ever leaves the device. No intermediary, including LifeLedger itself, can read plaintext medical data in transit or at rest.
- **IPFS for Tamper-Evident, Distributed Storage.** Encrypted record blobs are pinned to IPFS (via web3.storage), producing a content-addressed hash (CID). Because the CID is a cryptographic fingerprint of the content itself, any tampering with the underlying data changes the hash — making silent record tampering mathematically detectable.
- **On-Chain Hash Anchoring, Not On-Chain Medical Data.** The blockchain never stores raw or even encrypted medical content (this would be both expensive and a privacy anti-pattern). Instead, the smart contract stores only the IPFS CID, a timestamp, an issuer address, and an access-control reference — an immutable, auditable pointer to the encrypted truth.
- **Patient-Held Keys = No Central Honeypot.** Because decryption keys never leave the patient's device (or their guardian's, during custodianship), there is no central database an attacker can breach to expose thousands of patients at once. Compromising LifeLedger's infrastructure yields only encrypted, useless ciphertext.

Core Thesis:

Web2 healthcare fails because ownership and storage are collapsed into the same centralized entity. LifeLedger separates them: storage is decentralized (IPFS), access control is decentralized (smart contract), and ownership is cryptographic (the patient's private key) — not administrative.

3 Design Lineage & Key Differentiators

LifeLedger is a synthesis of two inputs, plus a set of original additions layered on top of both.

3.1 From the Healthchain Paper (Academic Base)

- The core privacy pattern: never put PII or medical content on-chain — only a hash/pointer. Actual data lives encrypted, off-chain, on IPFS.
- Fine-grained, **rule-based access control** — a doctor, pharmacist, or receptionist only sees what their role and the patient's grant permit, not the whole record by default.
- The idea of a **composite view** — a subset of the full record generated per request, not the entire history handed over wholesale.
- Provenance/audit trail — every read/write is a traceable, hashed transaction.

3.2 From the Core LifeLedger Design

- **ERC-4337 Account Abstraction** — no seed phrases. A parent logs in with email/passkey and a smart contract wallet is created for the child behind the scenes.
- **Progressive custody lifecycle** — Custodial (0–13) → Co-Managed (13–18) → Independent (18+), encoded directly into contract logic, not app-level policy.
- **Vaccination Soulbound Tokens (SBTs)** — non-transferable, unforgeable, un-losable proof of immunization minted straight to the patient's wallet.
- **Gasless UX** via a sponsoring paymaster, so parents/doctors never touch a gas token.

3.3 Our Additions on Top of Both

1. **Scoped access grants, not all-or-nothing.** Healthchain's "composite view" idea gets implemented on-chain as a *category bitmask*. When a patient grants a doctor access, they can restrict it to specific record types (e.g., "Vaccination + Allergy only," not the full history). This is strictly more granular than a full-access-per-visit grant.
2. **Break-glass emergency access.** Healthchain's related-work section explicitly flags that MedRec-style systems don't handle emergencies well — they do not allow a family member or verified clinician to obtain data access in an emergency without a live patient grant. LifeLedger adds a time-locked, mandatory-audit-logged emergency override so a verified ER doctor can access critical fields (allergies, blood type, current medications) without a live patient grant — with the access event permanently and visibly flagged on the patient's own activity feed.
3. **A lighter-weight chain choice.** Healthchain used permissioned Hyperledger Fabric; LifeLedger deliberately uses a public L2 testnet (Polygon Amoy or Base Sepolia) instead — because SBTs, ERC-4337 smart accounts, and free public paymasters are EVM/L2-native and dramatically cheaper to build against than standing up a permissioned Fabric network from scratch.

These three additions are treated as differentiators to layer onto the core loop once it works end-to-end, rather than as blockers for a first working version — see the priority tags throughout Section 4 and the open decisions in Section 11.

4 Complete Feature Set

Every feature below is tagged by priority:

CORE — the system doesn't demo without it

SHOULD-HAVE — strong differentiator, build if time allows

STRETCH — nice pitch line, cut first if squeezed

LAST HOOK — deliberately positioned as the very last thing to build, after the rest of the pipeline is solid

4.1 Identity & Custody Lifecycle

- **CORE** Guardian creates a child's identity with just email/passkey login — smart contract wallet deployed behind the scenes via Account Abstraction, no seed phrase ever shown.
- **CORE** Guardian has full signing authority while `custodyStatus == Custodial`.
- **CORE** At age 13 (or demo-toggled), `enableCoManagement()` registers the teen's own login as a second signer; status moves to `CoManaged`.
- **SHOULD-HAVE** Dual-control (2-of-2) requirement for *sensitive* teen-phase actions (granting a new doctor long-term access, exporting data), while routine actions (viewing records, accepting a vaccination SBT) need only the teen's signature.
- **CORE** At 18, `claimIndependence()` is **permissionlessly callable by anyone** (the app, a keeper bot, or the user) — it verifies the age threshold on-chain, revokes the guardian's signer key, and re-points the decryption key chain so only the now-adult patient can derive access going forward. Nobody — not the LifeLedger team, not the original guardian, not the original hospital — can reverse it.

4.2 Vaccination Soulbound Tokens (SBTs)

- **CORE** Verified clinicians (whitelisted in a `DoctorRegistry`) mint a non-transferable ERC-5192-style token directly to the patient's wallet on every vaccination.
- **CORE** Each SBT encodes vaccine type, batch number (for recall tracing), administering clinician, and timestamp.
- **SHOULD-HAVE** QR-based instant verification of a vaccination SBT by any third party (school, airline) without needing full record access.

4.3 The Doctor Visit Flow (the demo's centerpiece)

- **CORE** Patient/guardian generates a single-use, time-limited QR code encoding a signed access grant.
- **CORE** Doctor's clinic portal scans it, submits the signature, decrypts only what's authorized, and can add a new encrypted record anchored back on-chain.
- **CORE** Access **automatically expires** (default 24h) — the doctor retains zero standing access after the window closes.

- SHOULD-HAVE Scoped grants: the QR/grant transaction includes a record-category bitmask so the doctor only decrypts, e.g., allergy + past-diagnosis records, not everything.
- STRETCH Break-glass emergency access path for unscheduled ER visits.

4.4 Encryption & Storage

- CORE Every record is encrypted **client-side**, before it ever leaves the browser — nobody, including the LifeLedger team, can read plaintext in transit or at rest.
- CORE Encrypted blobs are pinned to IPFS; only the resulting content hash (CID) — plus timestamp, issuer address, and record-type tag — is written on-chain.
- SHOULD-HAVE Envelope encryption: each record gets its own random AES-256-GCM key, which is then individually wrapped (ECIES) for each authorized party's public key — so granting a new doctor access is a cheap key-wrap operation, not a full re-upload.

4.5 Access Control & Consent

- CORE Patient (or guardian, during the custodial phase) is the sole grantor/revoker of access — `grantAccess()`, `revokeAccess()`, gated by an `onlyController` modifier that respects the current custody phase.
- SHOULD-HAVE Role-based *and* scope-based rules (doctor vs. pharmacist vs. receptionist see different slices), inspired directly by Healthchain's ACL model, implemented here as an on-chain enum + bitmask instead of an off-chain ruleset.

4.6 GDPR — "Crypto-Shredding" Right to Erasure

- SHOULD-HAVE "Deletion" means destroying the per-record symmetric key everywhere LifeLedger controls it, plus issuing an IPFS unpin request. The on-chain CID becomes a pointer to content that is cryptographically unrecoverable and, over time, physically gone from the network — satisfying erasure without needing (or being able) to rewrite the chain.

4.7 Provenance / Audit Trail

- CORE Every access grant, revocation, and record addition emits an on-chain event, giving the patient a human-readable activity feed of exactly who touched their record and when.

4.8 Adult Onboarding & Legacy Record Migration

Covers the case of an adult who has, up to now, only ever had physical/paper records and wants to switch over.

- SHOULD-HAVE `createIndependentIdentity()` — a dedicated factory entry point for adults signing up directly: `custodyStatus = Independent` from creation, no guardian phase, no `claimIndependence()` detour needed.
- SHOULD-HAVE `addSelfAttestedRecord(cid, recordType)` — the patient adds their own historical records (old discharge summaries, allergy lists, prescriptions) without a doctor present. Same client-side-encrypt → IPFS → anchor pipeline as every other record.

- SHOULD-HAVE Provenance tagging on every `RecordRef`: `SelfReported` vs. `ClinicianVerified`. Self-reported entries are visibly badged as such in the UI — they carry less trust weight than a record a doctor wrote live.
- STRETCH `attestRecord(recordId)` — callable only by a registered doctor, upgrades a `SelfReported` record to `ClinicianVerified` once they've reviewed the original paper document at a real visit. The original event date is preserved separately from the attestation date.
- STRETCH Retroactive vaccination attestation: a doctor who reviews a physical vaccination card can mint an SBT carrying `attestedRetroactively = true` and the true historical date — distinct from a live-administered SBT.
- STRETCH Client-side OCR (e.g. Tesseract.js, free/OSS, runs entirely in-browser) to help transcribe a photographed paper record into structured fields before encryption — keeps "nothing plaintext ever leaves the device" intact even during bulk migration.

4.9 Prescriptions & Pharmacist Flow

- SHOULD-HAVE `PharmacistRegistry.sol` — whitelist of verified pharmacists, mirrors `DoctorRegistry`.
- SHOULD-HAVE Prescription records carry a `PrescriptionStatus` enum (`Issued`, `PartiallyFilled`, `Fulfilled`, `Expired`, `Cancelled`) and a `refillsRemaining` counter — genuinely protects against a script being filled twice, a real gap in paper prescriptions today.
- SHOULD-HAVE `dispensePrescription(prescriptionId)` — callable only by a whitelisted pharmacist holding a valid, scoped (`Prescription`-only) access grant. Decrements `refillsRemaining`, flips status to `Fulfilled` at zero.
- SHOULD-HAVE Same QR-based, time-boxed grant pattern as the doctor visit flow — patient shows a QR at the pharmacy counter, access lapses right after.

4.10 E-Referrals & Out-of-Network Sharing

- SHOULD-HAVE Doctor-to-doctor e-referrals: a referral record type linking `referringDoctorId` → `referredDoctorId`, which also auto-extends a scoped access grant to the receiving doctor for the referenced record categories.
- SHOULD-HAVE `grantReadAccess(anyPublicKey, scope, expiry)` — read-only sharing with a party *outside* any registry (a specialist abroad for a second opinion, a family member) via pure envelope-encryption re-wrap. No registry whitelisting, no write privilege.

4.11 Additional Record Granularity

- STRETCH Split `LabReport` into distinct **lab order** and **lab result** record types instead of conflating the two.
- STRETCH Appointment booking / receptionist role, modeled on Healthchain's receptionist ACL — flagged as scope creep for a first demo since it's closer to hospital-operations software than a health record, but easy to bolt on if time allows.

4.12 Insurance Claim Verification — The Last Hook

LAST HOOK Deliberately positioned as the final feature to build, after the rest of the pipeline is solid. It's a strong closing pitch line ("we thought about the business model"), not something the core demo depends on.

Methodology:

1. `InsurerRegistry.sol` — a third whitelist, mirroring `DoctorRegistry` / `PharmacistRegistry`, for verified insurer addresses.
2. Patient grants an insurer a scoped access grant restricted to the *specific* claim-relevant record ID(s) — not a category, not the full history. A claim for a surgery shouldn't unlock mental health notes.
3. A read-only `verifyClaim(patientAddress, recordId)` view function lets the insurer's off-chain claims system confirm three things, without seeing anything beyond that one record: (a) the record's CID and issuer signature are genuinely anchored on-chain and untampered; (b) the insurer currently holds a valid, unexpired grant covering that exact record; (c) optionally, that the record is `ClinicianVerified`, not `SelfReported`, since a claim probably shouldn't be payable off a patient-typed entry.
4. This is **verification, not payout**. The contract confirms a claimed diagnosis/procedure genuinely exists in the patient-controlled record and hasn't been altered — it does not move money or adjudicate the claim. Automated conditional payout stays a true future-scope item (Section 13), not something designed into the prototype.
5. Reuses the existing provenance pattern (Section 4.6): `ClaimVerificationRequested` / `ClaimVerified` events mean the patient sees "Insurer X verified claim for record Y on date Z" in their own activity feed — same audit trail as every other access, just with an insurer as the actor.

5 Core User Journeys

LifeLedger's signature innovation is treating identity ownership as something that evolves over a human lifetime, encoded directly into smart contract logic. This is modeled as three sequential custody phases per patient identity, plus flows for every other role that interacts with the record.

5.1 Phase 1 — Custodial Management (Birth to Age 13)

When a child is born, the attending hospital or the parent initiates a LifeLedger Identity Contract for the child.

- **Wallet Creation via Account Abstraction.** The mother logs into LifeLedger with just her email or biometrics (Privy/Biconomy-powered ERC-4337 smart account — no seed phrases, no browser extensions). Behind the scenes, this creates a smart contract wallet for the child, with the mother's authentication credential registered as the sole guardian signer.
- **Guardian-Managed Record.** The mother can add records on the child's behalf: birth weight/length, allergies, early screening results, and family medical history — all encrypted client-side and uploaded to IPFS, with hashes anchored on-chain under the child's contract.
- **Vaccination Soulbound Tokens (SBTs).** When the child receives a vaccination, the administering pediatrician (an address whitelisted as a `verifiedIssuer` in LifeLedger's Doctor Registry) mints a non-transferable ERC-5192-style Soulbound Token directly to the child's wallet. Each SBT encodes the vaccine type, batch/lot number (for recall tracing), administering clinician, and date — instantly and permanently verifiable, and impossible to lose, forge, or leave at home.
- **Full Guardian Authority.** During this phase, the smart contract's access-control logic treats the guardian's signature as fully authoritative for `grantAccess()`, `addRecord()`, and `revokeAccess()` calls — the child has no independent signing capability yet, mirroring real-world legal guardianship norms.

5.2 Phase 2 — Co-Management (Age 13 to 18)

As the child becomes a teenager, LifeLedger transitions the account into a dual-control model rather than an abrupt handover.

- **Teen Device Onboarding.** The teen installs the LifeLedger app and authenticates via their own email/passkey. This does not create a new wallet — it registers a second signer key against the same smart contract wallet, with a permission tier below full ownership.
- **Co-Signature for Sensitive Access.** Certain actions — such as granting a new doctor access to the full historical record, or exporting data outside the platform — require a threshold signature: both the guardian and the teen must approve within a configurable time window (implemented via a lightweight 2-of-2 multisig pattern layered on the ERC-4337 smart account's `validateUserOp`).
- **Growing Autonomy, Bounded Risk.** Routine actions (viewing records, accepting a new vaccination SBT, sharing a QR-based access token for a single doctor visit) can be approved by the teen alone, since these carry low risk and high day-to-day utility. This mirrors adolescent medical consent norms in most jurisdictions and builds the teen's familiarity with self-custody before the stakes are permanently theirs alone.

- **Guardian Visibility Persists.** The mother retains full read access and a real-time notification feed throughout this phase — nothing is hidden, but control is gradually, structurally shared.

5.3 Phase 3 — Independence (Age 18+)

This is LifeLedger's flagship mechanism, and the centerpiece of the demo.

- **Autonomous On-Chain Execution.** Each Identity Contract stores an immutable `dateOfBirth` value, set once at creation and attested by the issuing hospital/guardian. A public, permissionless function — `claimIndependence()` — becomes callable by anyone (typically the app itself, via a keeper job, or the user themself) the instant `block.timestamp >= dateOfBirth + 18 years`.
- **What `claimIndependence()` Does, Atomically:**
  1. Verifies the 18-year threshold has genuinely elapsed (no early execution possible — enforced by the EVM itself, not application logic).
  2. Permanently revokes the guardian's signer key from the smart account's owner set.
  3. Re-encrypts the master symmetric key reference so that only the (now-adult) primary signer can derive access to historical records going forward — the guardian's copy of the key-encryption-key is invalidated.
  4. Emits an `IndependenceClaimed` event, which the frontend uses to trigger a celebratory, irreversible "Welcome to full ownership" UX moment.
  5. Updates the on-chain `custodyStatus` enum from `CoManaged` to `Independent` — this status is publicly readable and forms the basis of the access-control checks in every other function.
- **No Central Party Can Reverse This.** Not the LifeLedger team, not the original guardian, not even the original hospital can undo this transition. It is enforced by contract logic, not by policy — which is precisely the point.

5.4 The Doctor Visit Flow

This is the moment-to-moment interaction that makes LifeLedger usable, not just architecturally elegant. It is the flow the live demo centers on.

1. **QR Scan for Time-Boxed Access.** At check-in, the patient (or guardian) opens the LifeLedger app and generates a single-use QR code. This QR encodes a signed, time-limited access grant — a `grantAccess(doctorAddress, expiryTimestamp, scope)` transaction (sponsored/gasless via the AA paymaster) authorizing this specific doctor's wallet address to decrypt this specific patient's record for a short window (default: 24 hours), optionally restricted to a record-category bitmask.
2. **Doctor Scans & Reads History.** The clinic's LifeLedger portal scans the code, submits the corresponding signature, and — if valid — the frontend fetches the relevant IPFS CIDs from the on-chain record index and decrypts them locally using a key the patient's contract released specifically to the doctor's public key (via asymmetric re-encryption, not by exposing the patient's private key itself).
3. **Doctor Writes a New Diagnosis.** The doctor enters diagnosis notes, prescriptions, or lab orders into a structured form.

4. **Client-Side Encryption Before Upload.** The moment "Save" is pressed, the browser generates a fresh AES-256-GCM key, encrypts the record with it, then wraps that AES key with the patient's public key via ECIES (`eth-crypto`). Nothing plaintext ever leaves the browser.
5. **Upload to IPFS.** The encrypted blob is pushed to web3.storage, returning a content-addressed CID.
6. **Hash Anchored On-Chain.** The doctor's wallet (gas-sponsored) calls `addRecord(cid, recordType)`, which is only valid because their access grant from Step 1 is still active — enforced by an `onlyAuthorizedIssuer` modifier checking the on-chain access-control mapping.
7. **Automatic Access Revocation.** When `expiryTimestamp` passes (or immediately after the visit, if the patient manually calls `revokeAccess()`), the doctor's decryption capability lapses. They retain zero standing access to the patient's historical or future records — every future visit requires a fresh, explicit grant.

Why this matters for the pitch:

Step 7 is the detail that separates LifeLedger from "just another patient portal." Access is ephemeral and re-negotiated by default, inverting the Web2 norm where any provider you've ever seen retains standing access to your chart forever.

5.5 Adult Migrating from Paper Records

Covers a patient who has never used LifeLedger before and has only ever had physical records.

1. Signs up directly via `createIndependentIdentity()` — no guardian phase, `custodyStatus = Independent` from day one.
2. Uses `addSelfAttestedRecord()` to enter historical vaccination cards, discharge summaries, and allergy lists — photographed and optionally OCR-assisted, but still encrypted client-side before anything uploads.
3. Those entries are visibly badged `SelfReported`.
4. At any real visit afterward, the attending doctor can review the physical original and call `attestRecord()` to upgrade it to `ClinicianVerified`, while the original historical date is preserved separately from the attestation date.

5.6 Pharmacist

Registered in `PharmacistRegistry`, the pharmacist follows a short, tightly scoped flow: the patient shows a QR at the counter, scoped to `Prescription` records only; the pharmacist calls `dispensePrescription()`, which checks and decrements `refillsRemaining`, flipping status to `Fulfilled` at zero; access then lapses immediately, with no standing view into anything else in the patient's history.

5.7 Receptionist (Stretch)

Read-only, appointment-metadata-scoped access only, if appointment booking gets built — modeled on Healthchain's role-based ACL pattern.

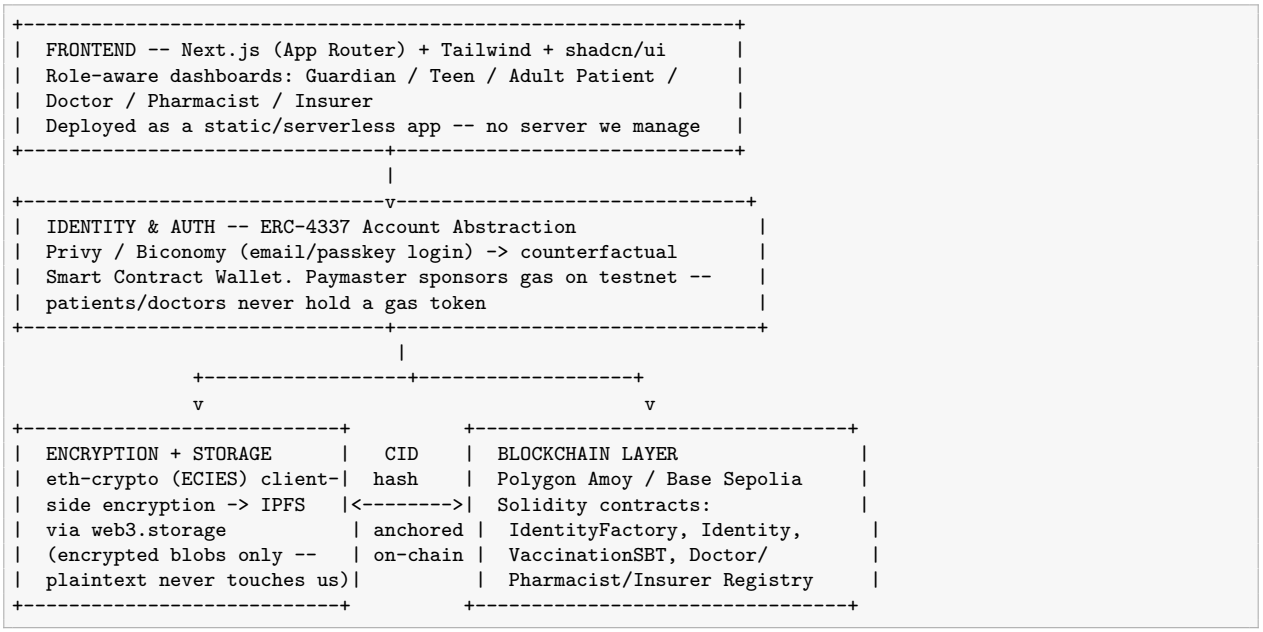
5.8 Insurer

Registered in `InsurerRegistry` → holds a per-record (not per-category) scoped grant for exactly the record(s) tied to a specific claim → calls `verifyClaim()` to confirm the record is genuinely anchored, grant-covered, and (optionally) clinician-verified → the verification itself is logged to the patient's own activity feed, same as any other access.

6 Technical Architecture & Tech Stack

6.1 System Architecture Overview

LifeLedger is a four-layer system — a Next.js frontend, an Account-Abstraction identity/auth layer, a client-side encryption + IPFS storage layer, and a Solidity smart contract layer on an L2 — and, deliberately, **no traditional backend server**. This keeps the whole thing on free-tier infrastructure and removes an entire class of "who's paying for hosting" problems. No layer trusts any other layer with plaintext data or unchecked authority.



6.2 How a Record Moves Through the System

This is the part that's easiest to get fuzzy on, so it is worth being precise — this is the sequence for a doctor adding a new diagnosis during a visit:

1. **Grant.** Patient's app generates a QR encoding a signed `grantAccess(doctorAddress, expiry, scopeBitmask)` call. This is a gasless UserOperation — the paymaster covers it.
2. **Scan & verify.** The clinic portal scans the QR, submits it, and the contract checks the signature and custody-phase authority (`onlyController`).
3. **Doctor writes.** Doctor fills a structured form (diagnosis notes, prescription, etc.) in the portal.
4. **Client-side encryption.** The instant "Save" is pressed, the browser generates a fresh AES-256-GCM key, encrypts the record with it, then wraps that AES key with the *patient's* public key via ECIES (`eth-crypto`). Nothing plaintext ever leaves the browser.
5. **Upload.** The encrypted blob goes to IPFS via `web3.storage`, returning a content-addressed CID.
6. **Anchor.** Doctor's (gas-sponsored) wallet calls `addRecord(cid, recordType)` — only valid because the access grant from step 1 is still active (`onlyAuthorizedIssuer` modifier).

7. **Expiry.** When `expiryTimestamp` passes (or the patient manually revokes), the doctor's decryption capability lapses. Every future visit needs a fresh, explicit grant — there is no standing access, ever.
8. **Patient reads.** Later, the patient's app fetches the CID from the on-chain record index, downloads the encrypted blob from IPFS, and decrypts it locally using their own private key (held inside their AA smart account's associated keypair, never exported to the LifeLedger team).

That's the whole loop. Everything else in the app (dashboards, QR UI, the 18th-birthday transition) is UI wrapped around this core sequence plus the custody-transition logic.

6.3 Record Types & Provenance

The `recordType` field on every anchored record is deliberately granular, to properly support hospital reports, prescriptions, and referrals:

```
enum RecordType {
    Diagnosis, LabOrder, LabResult, Imaging, ClinicalNote,
    HospitalAdmission, DischargeSummary, SurgicalReport,
    Prescription, AllergyRecord, ReferralRecord, InsuranceClaim
}

enum ProvenanceSource { ClinicianVerified, SelfReported }
```

Every `RecordRef` carries both fields. `ClinicianVerified` records come out of the doctor-visit flow or a hospital/pharmacist writing directly during a visit; `SelfReported` records come from an adult migrating in paper history and stay visibly badged as such until a clinician reviews and attests them.

6.4 Frontend

- **Next.js 14 (App Router)** — server components for public/marketing pages, client components for wallet-connected dashboards; route-based role gating (`/guardian`, `/patient`, `/doctor`, `/clinic-portal`, `/pharmacist-portal`).
- **Tailwind CSS + shadcn/ui** — accessible, themeable component primitives (dialogs, forms, data tables) so the team can move fast without hand-rolling UI kit primitives.
- **wagmi + viem** — typed contract read/write hooks against the deployed Solidity ABI, layered underneath the Privy/Biconomy smart-account provider.
- **QR generation/scanning** — `qrcode.react` for generation, `html5-qrcode` for the clinic portal's camera-based scan flow.

6.5 Wallet, Auth & Account Abstraction

- **ERC-4337 Smart Contract Wallets** via Privy (primary) or Biconomy (fallback). Users authenticate with email or passkey; a `SmartAccount` is deployed counterfactually (no gas cost until first use) and becomes each user's on-chain identity.
- **Paymaster-Sponsored Transactions.** A project-funded paymaster sponsors gas for record-adding and access-granting UserOperations, so parents and doctors never need to hold native gas tokens — critical for real-world healthcare adoption.

- **Guardian/Teen/Adult Signer Sets.** LifeLedger layers a lightweight custom ownership module on top of the smart account, tracking `guardianSigner`, `coSigner` (teen), and `custodyStatus`, rather than relying on a single fixed owner key — this is the technical foundation for the entire custody lifecycle.

6.6 Decentralized Storage & Encryption

- **eth-crypto for ECIES Encryption.** Each record is encrypted client-side against the recipient's public key (derived from their wallet) before leaving the browser — asymmetric encryption means only the intended holder of the matching private key can decrypt, with no shared-secret management burden.
- **web3.storage (IPFS/Filecoin).** Encrypted blobs are pinned via the web3.storage API, yielding a content-addressed CID used as the canonical on-chain reference. Filecoin-backed persistence protects against IPFS pin-availability decay over a lifetime-scale timeline.
- **Envelope Encryption for Access Sharing.** Rather than re-encrypting the entire record for every new doctor, LifeLedger encrypts each record once with a per-record symmetric key (AES-256-GCM), then encrypts that key individually for each authorized party's public key — access grants become cheap, small on-chain operations instead of expensive re-uploads.

6.7 Blockchain Layer

Chain: Polygon Amoy testnet (fallback: Base Sepolia). Both offer sub-cent transaction costs and roughly 2-second finality, essential for a UX where doctors and parents interact with the chain dozens of times without feeling "blockchain friction." Base is preferred if the team wants Coinbase Smart Wallet / OP Stack ecosystem tie-ins for the pitch; Polygon is preferred for broader existing healthcare/RWA precedent.

Language & tooling: Solidity ^0.8.24, built with Foundry for fast iteration, fuzz testing, and gas profiling.

6.7.1 Key Smart Contracts & Functions

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

/// @title LifeLedgerIdentity
/// @notice One deployed instance per patient. Deployed via a minimal-proxy
/// (ERC-1167) IdentityFactory to keep per-patient deployment cheap.
contract LifeLedgerIdentity {

    enum CustodyStatus { Custodial, CoManaged, Independent }

    address public guardianSigner;   // parent/guardian address, zero after claim
    address public coSigner;         // teen co-signer, set at age 13
    address public owner;            // the patient themself, always set
    uint256 public dateOfBirth;      // unix timestamp, set once at creation
    CustodyStatus public custodyStatus;

    struct RecordRef {
        bytes32 cid;        // IPFS CID (encoded)
        address issuer;     // doctor/hospital address that wrote it
        uint256 timestamp;
        uint8   recordType; // enum: Diagnosis, Lab, Imaging, Note...
        uint8   provenance; // enum: ClinicianVerified, SelfReported
    }
```

```
    RecordRef[] public records;
    mapping(address => uint256) public accessExpiry;   // doctor => expiry
    mapping(address => uint256) public accessScope;     // doctor => category bitmask

    event AccessGranted(address indexed doctor, uint256 expiry, uint256 scope);
    event AccessRevoked(address indexed doctor);
    event RecordAdded(address indexed issuer, bytes32 cid, uint8 recordType);
    event IndependenceClaimed(address indexed newOwner, uint256 timestamp);

    modifier onlyController() {
        require(
            msg.sender == owner ||
            (custodyStatus == CustodyStatus.Custodial && msg.sender == guardianSigner) ||
            (custodyStatus == CustodyStatus.CoManaged &&
                (msg.sender == guardianSigner || msg.sender == coSigner)),
            "LifeLedger: not authorized to control this identity"
        );
        _;
    }

    modifier onlyAuthorizedIssuer(address doctor) {
        require(accessExpiry[doctor] >= block.timestamp, "LifeLedger: access expired/unset");
        _;
    }

    /// @notice Guardian or patient grants a time-boxed, optionally scoped
    /// access window (scope = a record-category bitmask; 0 = full access).
    function grantAccess(address doctor, uint256 durationSeconds, uint256 scope)
        external
        onlyController
    {
        accessExpiry[doctor] = block.timestamp + durationSeconds;
        accessScope[doctor] = scope;
        emit AccessGranted(doctor, accessExpiry[doctor], scope);
    }

    /// @notice Explicit early revocation -- e.g. patient ends the visit early.
    function revokeAccess(address doctor) external onlyController {
        accessExpiry[doctor] = 0;
        emit AccessRevoked(doctor);
    }

    /// @notice Doctor anchors a new encrypted record's IPFS pointer on-chain.
    /// Requires an active access grant covering this record's category.
    function addRecord(bytes32 cid, uint8 recordType, uint8 provenance)
        external
        onlyAuthorizedIssuer(msg.sender)
    {
        records.push(RecordRef(cid, msg.sender, block.timestamp, recordType, provenance));
        emit RecordAdded(msg.sender, cid, recordType);
    }

    /// @notice Called once the teen turns 13 -- registers the second signer
    /// and moves the identity from Custodial into CoManaged.
    function enableCoManagement(address teenSigner) external {
        require(msg.sender == guardianSigner, "LifeLedger: only guardian can enroll co-signer");
        require(block.timestamp >= dateOfBirth + 13 * 365 days, "LifeLedger: too young for co-management");
        coSigner = teenSigner;
        custodyStatus = CustodyStatus.CoManaged;
    }

    /// @notice Permissionless -- callable by anyone (typically a keeper bot or
    /// the user themself) once the patient has genuinely turned 18.
    function claimIndependence() external {
        require(custodyStatus != CustodyStatus.Independent, "LifeLedger: already independent");
        require(block.timestamp >= dateOfBirth + 18 * 365 days, "LifeLedger: not yet 18");
        guardianSigner = address(0);
```

```
        coSigner = address(0);
        custodyStatus = CustodyStatus.Independent;
        emit IndependenceClaimed(owner, block.timestamp);
    }
}

/// @title VaccinationSBT
/// @notice ERC-5192-style soulbound token minted by verified clinicians.
/// Non-transferable by design -- a vaccination record cannot be
/// sold, gifted, or forged onto another identity.
contract VaccinationSBT {
    mapping(address => bool) public verifiedIssuers; // Doctor Registry lookup

    struct Vaccination {
        string vaccineType;
        string batchNumber;
        address administeredBy;
        uint256 timestamp;
        bool attestedRetroactively;
    }

    mapping(uint256 => Vaccination) public vaccinations;
    mapping(uint256 => address) public ownerOf; // patient identity contract
    uint256 public nextTokenId;

    event VaccinationMinted(uint256 indexed tokenId, address indexed patient, string vaccineType);

    function mint(address patient, string calldata vaccineType, string calldata batchNumber) external {
        require(verifiedIssuers[msg.sender], "LifeLedger: not a verified clinician");
        uint256 tokenId = nextTokenId++;
        vaccinations[tokenId] = Vaccination(vaccineType, batchNumber, msg.sender, block.timestamp, false);
        ownerOf[tokenId] = patient;
        emit VaccinationMinted(tokenId, patient, vaccineType);
    }

    // transferFrom() intentionally omitted -- soulbound, non-transferable.
}
```

6.7.2 Extended Contract Additions (the "Tadka" Layer)

The functions below extend the core contracts above to cover adult onboarding, pharmacist dispensing, out-of-network sharing, and insurance verification. They are documented here as function-level specifications so Track A can implement them once the core loop is stable.

Function	Purpose
`createIndependentIdentity()`	Factory entry point for an adult signing up directly; sets `custodyStatus = Independent` from creation, skipping the guardian phase entirely.
`addSelfAttestedRecord(cid, recordType)`	Lets an `Independent` patient add their own historical record without a doctor present; tags it `SelfReported`.
`attestRecord(recordId)`	Callable only by a registered doctor; upgrades a `SelfReported` record to `ClinicianVerified` after in-person review, preserving the original event date separately from the attestation date.
`dispensePrescription(prescriptionId)`	Callable only by a whitelisted pharmacist holding a valid, `Prescription`-scoped grant; decrements `refillsRemaining` and flips status to `Fulfilled` at zero.

Function	Purpose
`grantReadAccess(publicKey, scope, expiry)`	Read-only sharing with any party outside the doctor/pharmacist/insurer registries (e.g. a specialist abroad), via pure envelope-encryption re-wrap. No write privilege.
`verifyClaim(patientAddress, recordId)`	Read-only view function for a whitelisted insurer; confirms a specific record is genuinely anchored, covered by a valid grant, and untampered — without exposing anything beyond that one record.

6.7.3 Supporting Infrastructure

- `IdentityFactory.sol` — deploys gas-cheap minimal-proxy (EIP-1167) clones of `LifeLedgerIdentity` per patient, and maintains the canonical registry mapping real-world identity claims (via guardian attestation at creation) to on-chain addresses.
- `DoctorRegistry.sol` — a whitelist/attestation contract (owned by a multisig representing partnered hospital networks during the pilot phase) that gates who can call `addRecord()` and mint `VaccinationSBT`s — preventing spam or malicious record injection.
- `PharmacistRegistry.sol` — mirrors `DoctorRegistry`; whitelist of verified pharmacists gating `dispensePrescription()`.
- `InsurerRegistry.sol` — mirrors `DoctorRegistry` / `PharmacistRegistry`; whitelist of verified insurer addresses gating `verifyClaim()`. Built last, only once the rest of the pipeline is solid.
- **Keeper Automation** — an off-chain keeper (Chainlink Automation or Gelato) periodically calls `claimIndependence()` for any identity that has crossed the 18-year threshold, so the transition genuinely happens automatically rather than depending on the user remembering to click a button, with the function remaining permissionlessly callable by anyone as a trustless fallback.

6.8 Full Tech Stack

No paid service is required anywhere in this stack for a small-scale prototype.

Layer	Choice	Why / Free-tier notes
Chain	Polygon Amoy testnet (fallback: Base Sepolia)	Sub-cent gas, ˜2s finality, free faucet. Fine to switch to Base Sepolia if Coinbase Smart Wallet tie-ins matter more for the pitch.
Smart contracts	Solidity ˆ0.8.24 + Foundry	Foundry is fully free/open-source, fast fuzz testing, no license cost.
RPC provider	Alchemy (free tier)	Generous free compute-unit allowance, works for both Amoy/Base Sepolia.
Account Abstraction	Privy (embedded wallets, free dev tier)	Email/passkey login → ERC-4337 smart account, no seed phrase UX.

Layer	Choice	Why / Free-tier notes
Gas sponsorship	Biconomy paymaster (free tier) or Alchemy Account Kit's bundled paymaster	Sponsors testnet UserOperations so nobody needs a gas token.
Client-side encryption	eth-crypto (npm, free/OSS)	ECIES envelope encryption, works entirely in-browser.
Off-chain storage	web3.storage (free tier via w3up)	Free IPFS/Filecoin pinning, generous quota for a prototype's data volume. Pinata is a fine fallback.
Frontend	Next.js 14 (App Router)	Free to build/run; deploys free on Vercel.
Styling/UI	Tailwind CSS + shadcn/ui	Free component primitives, fast to theme.
Contract interaction	wagmi + viem	Typed hooks over the deployed ABI.
QR generation/scan	qrcode.react + html5-qrcode	Both free/OSS.
OCR (paper record transcription)	Tesseract.js (free/OSS, client-side)	Stretch feature for adult onboarding — transcription assist stays entirely in-browser, nothing plaintext leaves the device even mid-migration.
Automation (18th-birthday trigger)	Chainlink Automation or Gelato (free testnet tiers)	Keeps `claimIndependence()` genuinely automatic instead of relying on a manual click; contract remains permissionlessly callable as a trustless fallback either way.
Hosting	Vercel (frontend, free tier, custom domain supported)	No backend server needed — everything is client + chain + IPFS.
DNS	Existing domain, via Cloudflare (free DNS)	Point a subdomain at Vercel.
Version control	GitHub (free, private repo is fine for two collaborators)	Plus GitHub Actions free minutes for running Foundry tests on push, if desired.

7 Repository Structure & Parallel Work Split

7.1 Repository Layout

A single monorepo keeps the contract ABI and shared types flowing to the frontend without copy-pasting files around.

```
lifeledger/
|-- contracts/                  # Foundry project -- TRACK A
|   |-- src/
|   |   |-- LifeLedgerIdentity.sol
|   |   |-- IdentityFactory.sol
|   |   |-- VaccinationSBT.sol
|   |   |-- DoctorRegistry.sol
|   |   |-- PharmacistRegistry.sol
|   |   `-- InsurerRegistry.sol     # build last
|   |-- test/                   # unit + fuzz tests
|   |-- script/                 # deploy scripts
|   `-- foundry.toml
|
|-- apps/
|   `-- web/                    # Next.js frontend -- TRACK B
|       |-- app/
|       |   |-- guardian/
|       |   |-- patient/
|       |   |-- doctor/
|       |   |-- pharmacist-portal/
|       |   `-- clinic-portal/
|       `-- components/
|
|-- packages/                   # the shared contract between tracks
|   |-- contract-abi/           # ABI + deployed addresses.json
|   |-- shared-types/           # TS types: CustodyStatus, RecordType, etc.
|   |-- crypto/                 # encryptRecord() / decryptRecord() wrappers
|   |-- ipfs-client/            # uploadEncrypted() / fetchFromIPFS()
|   `-- mocks/                  # fake ABI/crypto/IPFS responses for early UI work
|
|-- docs/
|   `-- architecture.md
|
|-- .github/workflows/          # optional: run `forge test` on every push
|-- pnpm-workspace.yaml
|-- package.json
`-- README.md
```

Why this shape works for two people: the `packages/` folder is the contract between the two tracks. Track A owns the *implementation* of `crypto/` and `ipfs-client/`, but publishes a small, stable function signature early (`encryptRecord()`, `uploadEncrypted()`, etc.) that Track B can start calling — against a mock — from day one, without waiting for the real contracts to be deployed.

7.2 The Parallel Work Split

Track A — Blockchain, Cryptography & Chain Infra

- `LifeLedgerIdentity.sol` — custody-phase logic, `onlyController` modifier, `grantAccess` / `revokeAccess` / `addRecord`, `enableCoManagement`, `claimIndependence`
- `VaccinationSBT.sol` — non-transferable mint, verified-issuer gating
- `IdentityFactory.sol` — minimal-proxy (EIP-1167) clone deployment per patient

- `DoctorRegistry.sol` — whitelist/attestation for clinicians
- Foundry unit + fuzz tests, especially around access-control edge cases (expired grants, custody-phase transitions, replay of an old QR)
- Deploy scripts to Polygon Amoy (or Base Sepolia); export ABI + addresses into `packages/contract-abi`
- `packages/crypto`: envelope encryption (eth-crypto ECIES + AES-256-GCM) with a clean, documented function signature
- `packages/ipfs-client`: web3.storage upload/fetch wrapper
- Paymaster/AA backend config (Privy project setup, Biconomy paymaster policy) — published as something Track B just imports
- Keeper setup for `claimIndependence()` (Chainlink Automation / Gelato registration on testnet)
- Scoped-access design: the record-category bitmask, exposed as a shared enum in `packages/shared-types`
- `PharmacistRegistry.sol` + prescription status/refill logic on top of `addRecord`
- Adult onboarding path: `createIndependentIdentity()`, `addSelfAttestedRecord()`, `attestRecord()`, `ProvenanceSource` tagging
- `grantReadAccess()` for out-of-network sharing
- `InsurerRegistry.sol` + `verifyClaim()` — build **last**, only once the rest of the pipeline is solid

Track B — Full-Stack / Frontend

- Next.js app scaffold, role-based routing (`/guardian`, `/patient`, `/doctor`, `/clinic-portal`)
- Wallet connect / login UX using Privy's React components (against Track A's config)
- wagmi/viem hooks wired to `packages/contract-abi` for every read/write the UI needs
- Guardian dashboard: create-child-identity flow, add-record form, vaccination history view
- Doctor/clinic portal: QR scan flow, patient-history view respecting scoped access, new-record form
- QR generation (patient side) via `qrcode.react`
- Activity feed UI, driven by on-chain events
- The 18th-birthday transition demo toggle UI
- Tailwind/shadcn styling pass, loading/error/empty states, toasts
- Early screens built against `packages/mocks` so UI work isn't blocked on contracts being deployed
- Pharmacist portal: QR scan, `dispensePrescription()` call, refill-count UI

- Adult self-onboarding flow: direct signup form, "add historical record" upload UI, `SelfReported` / `ClinicianVerified` badges
- Optional: Tesseract.js-powered photo-to-form assist for paper record migration
- Minimal insurer-facing verification UI — built very last, after everything else works

Shared / Joint Ownership

- `packages/shared-types` — enums and interfaces both sides read from
- Deciding together: Polygon Amoy vs. Base Sepolia; Privy vs. an alternative AA provider; how much of the differentiator layer (scoped grants, break-glass) makes the first working cut vs. gets added once the core loop works

7.3 Git Workflow

- `main` — always demoable, protected
- `dev` — integration branch, both tracks merge here first
- Feature branches: `feat/contracts-<name>` for Track A, `feat/frontend-<name>` for Track B
- Small, frequent PRs against `dev`, even solo-reviewed — keeps merge conflicts in `packages/` small
- Conventional commits (`feat:`, `fix:`, `test:`) make it easy to scan history when integrating
- `.env.example` checked in (never the real `.env`) so either track can spin up the other's half locally

8 Deployment & Domain Setup

Since there's no backend server to host, deployment is just: frontend → Vercel, then point a domain at it.

1. Deploy `apps/web` to Vercel (free tier, connect the GitHub repo, auto-deploys on push to `main`).
2. In Vercel, add the domain (e.g. `app.yourdomain.com`) under Project → Settings → Domains.
3. In the DNS provider (Cloudflare free tier is easiest), add a `CNAME` record: `app` → `cname.vercel-dns.com`.
4. SSL is automatic and free on Vercel's side.
5. Optional: a second subdomain (e.g. `pitch.yourdomain.com` or the bare root) for a static one-page pitch/landing site, deployed the same way.

No server costs, no domain-registrar-specific lock-in beyond DNS records fully under the team's control.

Environment Variables (Template)

```
# apps/web/.env.example
NEXT_PUBLIC_CHAIN=polygon-amoy
NEXT_PUBLIC_ALCHEMY_RPC_URL=
NEXT_PUBLIC_PRIVY_APP_ID=
```

```
NEXT_PUBLIC_LIFELEDGER_IDENTITY_FACTORY_ADDRESS=
NEXT_PUBLIC_VACCINATION_SBT_ADDRESS=
NEXT_PUBLIC_DOCTOR_REGISTRY_ADDRESS=
NEXT_PUBLIC_WEB3_STORAGE_TOKEN=

# contracts/.env.example
ALCHEMY_RPC_URL=
DEPLOYER_PRIVATE_KEY=
BICONOMY_PAYMASTER_API_KEY=
```

9 Hurdle Mitigation

Judges and technical due-diligence investors will immediately probe the two hardest real-world objections to any self-custody health system. LifeLedger has concrete, code-level answers to both.

9.1 "What happens when someone loses their private key?"

Self-custody's classic failure mode — a lost seed phrase meaning permanently lost medical history — is precisely what Account Abstraction was invented to solve, and LifeLedger leans on it directly rather than treating it as an afterthought.

- **No Seed Phrases, Ever.** Because wallets are ERC-4337 smart accounts created via Privy/Biconomy's embedded auth, there is no 12-word phrase for a parent or patient to write down, lose, or have stolen. Authentication is email/passkey-based, backed by the provider's secure enclave/MPC infrastructure.
- **Social Recovery Guardians.** The smart account's owner set supports a configurable social recovery module: the patient can designate 2–3 trusted recovery guardians (a spouse, a second device, or — during the custodial phase — the original guardian and pediatric clinic). A quorum of recovery guardians can re-key the account to a new signer without any central party holding custody of funds or data.
- **Encrypted Key Escrow (Opt-In).** For users who want an additional safety net, the record's per-file symmetric keys can optionally be split via Shamir's Secret Sharing across the patient, a designated family member, and a LifeLedger-partnered custodian — reconstructable only with a quorum, never by any single party alone.
- **The Underlying Encrypted Data Is Never at Risk.** Even in a worst-case key loss with no recovery guardians configured, the IPFS-stored ciphertext remains intact and content-addressed — future key-recovery mechanisms (post-quantum re-encryption schemes, biometric re-derivation) can be layered in without any data migration, because the storage layer and the key layer are fully decoupled.

9.2 "How do you comply with GDPR's Right to Be Forgotten on an immutable blockchain?"

This is the objection every "blockchain + healthcare" pitch must answer convincingly, because naive designs (storing PII directly on-chain) are fundamentally incompatible with erasure requirements. LifeLedger's architecture was chosen specifically to sidestep this.

- **The Chain Never Stores Personal Data — Only Pointers.** On-chain state is limited to CIDs, timestamps, and addresses. A CID is a cryptographic hash — mathematically, it reveals nothing about the underlying content without the corresponding decryption key.
- **"Deletion" = Orphaning the Ciphertext, Not Erasing a Hash.** When a user exercises their right to erasure, LifeLedger does not attempt (and does not need) to remove the on-chain hash reference — doing so would be both technically difficult and unnecessary for compliance. Instead:
  1. The record's per-file symmetric key is permanently destroyed from every location LifeLedger controls.

2. web3.storage receives an unpin request, and the IPFS content is allowed to fall out of the network's replication set over time.
3. The remaining on-chain CID becomes a pointer to content that is now cryptographically unrecoverable and, over time, physically absent from the network — functionally equivalent to erasure, since a hash of nothing-you-can-access carries no personal data exposure.

- **Legal Precedent Alignment.** This "crypto-shredding" pattern (destroy the key, not the ciphertext) is explicitly recognized by data protection authorities (including guidance referenced by the UK ICO and various EU DPA opinions) as a valid technical implementation of erasure for encrypted data — LifeLedger's architecture was chosen specifically because this precedent exists.

Judge Q&A Prep:

If asked "isn't the hash itself still personal data under GDPR?" — the answer is that a CID with no retrievable, decryptable content behind it fails the "identifiability" test at the core of the GDPR's definition of personal data (Article 4(1)/Recital 26), since re-identification is not reasonably likely once the key and content are gone.

10 Open Design Decisions

The following are worth deciding together before building begins, rather than assuming a default:

- **Chain:** Polygon Amoy (broader healthcare/RWA precedent) vs. Base Sepolia (Coinbase Smart Wallet ecosystem tie-in for the pitch)?
- **AA provider:** Privy (simplest embedded-wallet DX) vs. Alchemy Account Kit (one vendor for both RPC + AA, less integration surface)?
- **Scope of the differentiator layer for v1:** build scoped/category access grants for the first demo, or ship full-access-per-grant first and layer scoping in once the core loop works end-to-end?
- **Emergency access:** worth the added contract complexity for a first demo, or a "future scope" pitch line instead?
- **Self-attested record trust in the UI:** should `SelfReported` records be visually de-emphasized in a doctor's view during a visit, or just badge-tagged and otherwise treated equally? Worth mocking up rather than deciding in the abstract.
- **Insurance methodology scope:** confirm the build is *verification only*, treating automated payout as explicitly out of scope for the prototype, so it doesn't quietly balloon into a second contract system.

11 Pitch & Demo Script

Delivered live alongside the actual running application — every claim below should be shown, not just told.

Opening visual:

A single, cracked hospital records folder icon, dissolving into a glowing chain of encrypted blocks.

> *"Healthcare is consistently one of the most breached industries on Earth. Somewhere right now, a mother is holding a soggy, half-legible vaccination card because it's the only proof her daughter was ever immunized. And somewhere else, a hospital's entire patient database just got locked by ransomware. This isn't a technology gap. It's an ownership gap. Patients don't own their health data — hospitals do. We think that's backwards. We built LifeLedger to fix it.*
>
> *Meet Alex. Alex doesn't exist yet — Alex is born today, right here in our demo. The moment Alex is born, their mother, Maria, logs into LifeLedger with nothing but her email. No seed phrase, no crypto wallet extension, no confusing words to write on a napkin. Behind the scenes, a smart contract wallet is created for Alex — instantly, gaslessly — using account abstraction. Maria is now the cryptographic guardian of Alex's lifelong medical identity.*
>
> (Live demo: create identity, show wallet creation in seconds.)
>
> *Over the next few years, Alex goes to the pediatrician. Every vaccination isn't scribbled on a card that'll get lost in a move — it's minted directly to Alex's wallet as a soulbound token. Unforgeable, un-losable, instantly verifiable by any school or country on Earth. Watch — Maria takes Alex in for a routine shot right now.*
>
> (Live demo: doctor mints a Vaccination SBT to Alex's wallet.)
>
> *Fast forward. Alex turns 13. They get their own phone, their own LifeLedger login — and now they co-manage their own health record alongside Maria. They're learning ownership before they need to hold all the responsibility.*
>
> *Now — the moment that makes LifeLedger different from every other digital health record on the market. Alex turns 18 today.*
>
> (Live demo: trigger the age-transition toggle. The `claimIndependence()` transaction fires on-screen in real time.)
>
> *No form. No hospital administrator approving a transfer. No company — including us — in the loop at all. A smart contract, written once and never touched since, just permanently and irreversibly handed Alex full cryptographic ownership of every single medical record of their entire life. Maria's access — gone, automatically. Alex is, for the first time, the sole owner of their own story.*
>
> *Some time later, Alex starts college in a new city. They walk into the campus health center for the first time ever. The doctor scans a QR code on Alex's phone.*
>
> (Live demo: QR scan → doctor instantly sees Alex's full, complete, eighteen-year medical history.)

Eighteen years of vaccination records, allergies, prior diagnoses — available in seconds, to a doctor who has never met Alex before, with zero paperwork, zero fax machines, zero 'please contact your previous provider.' And the second the visit ends, that access automatically expires. The doctor learns nothing more, ever again, unless Alex grants it again.

That's LifeLedger. Not a blockchain gimmick bolted onto healthcare — a fundamental rethink of who owns a human life's medical story, encoded not in policy, but in code that cannot be bribed, breached, or bureaucratically delayed. We're not asking hospitals to trust us with patient data. We're making it architecturally impossible for anyone to need to."

12 Future Scope

LifeLedger's initial build establishes the core sovereignty primitive — custody, encryption, access control, and the age-18 handover — plus the differentiator layer of scoped access, adult onboarding, pharmacist and insurer flows. The architecture is deliberately extensible toward a much larger long-term product surface:

- **AI Medical Copilots with Zero-Knowledge Context.** A patient-authorized AI assistant could query the encrypted record locally (client-side, using the patient's own decryption capability) to summarize history, flag drug interactions, or prep questions for an upcoming appointment — without the AI provider ever seeing plaintext data server-side.
- **IoT & Wearable Data Integration.** Continuous data streams (glucose monitors, smartwatches, heart-rate variability) could be periodically batched, encrypted, and anchored the same way clinical records are today — turning LifeLedger into a longitudinal record of lived health, not just clinical checkpoints.
- **Automated Insurance Claim Settlement.** Building on the verification-only layer in Section 4.12, patient-granted, scoped access could let insurers verify claim-relevant record hashes on-chain and move toward automated, condition-based smart contract payouts — reducing both fraud and multi-week reimbursement delays.
- **Cross-Border Portable Health Passports.** The same SBT-based vaccination proof mechanism that solves the "lost paper card" problem domestically extends naturally into instantly verifiable, privacy-preserving international travel health credentials.
- **DAO-Governed Research Data Marketplaces.** Patients could opt in to anonymized, compensated data-sharing pools for medical research — with consent, revocation, and payment all enforced on-chain, replacing today's opaque and often uncompensated data-broker ecosystem.